Competitive Programming

Section 44 Editorial: String Algorithms II

Competitive Programming Course

2026-10-24

Table of contents

1 Editorial overview 3

2 A. Trie Prefix Count 4

3 B. Palindrome Queries 8

4 C. Distinct Substrings 13

5 D. Unique Prefix Lengths 19

6 Testing discussion 23

7 What to carry forward 24

1 Editorial overview

The lesson develops the reusable structures; this editorial applies their invariants to the four local problems. Trie operations are in Chapter 2, rolling hash in Chapter 3, and the suffix automaton in Chapter 4, including its groups of ending positions, suffix links, and clone operations.

Problem	Local constraints	Main idea
A. Trie Prefix Count	At most 200000 dictionary characters and 200000 query characters	Count dictionary entries passing through each prefix node
B. Palindrome Queries	`n, q <= 200000`	Compare forward and mirrored reverse hashes
C. Distinct Substrings	`n <= 200000`	Sum the sizes of the final SAM groups
D. Unique Prefix Lengths	At most 200000 input characters	Find the first unique node on each word's trie path

The lesson contains reusable structure implementations. Each exercise below connects their invariants to its own algorithm, sample, proof, and complexity, then gives a complete C++ and Python program. The listings include the tested reference files directly, so the printed solutions and the local judge's reference code stay identical.

2 A. Trie Prefix Count

2.1 Restatement and constraints

For every query string, count the dictionary entries beginning with it. Entries are counted separately, so duplicate words contribute more than once. The whole word is one of its prefixes; queries never modify the dictionary.

Let `W` be the total dictionary length and `P` the total query length. Both are at most `200000`. Scanning the whole dictionary for each query can repeat far too much work. A trie shares the work for common prefixes.

2.2 Observation and algorithm

The characters on the root-to-node path identify one prefix. Store `pass[v]`, the number of inserted dictionary entries whose path visits node `v`. Increment it for every entry, including another copy of an equal word.

A query walks its characters from the root. If an edge is missing, no word has that prefix, so the answer is zero. Otherwise the final node's `pass` is exactly the required count. Count words passing through the node, not only words ending there.

```
create an empty trie
for each dictionary entry:
    insert it, incrementing the pass counts on its path

for each query prefix:
    walk its characters from the root
    if an edge is missing, answer 0
    otherwise, answer the final node's pass count
```

2.3 Sample walkthrough

Insert `apple`, `ape`, `bat`, and `bar`:

Query	Entries sharing the prefix	Answer
`ap`	`apple`, `ape`	2
`app`	`apple`	1
`b`	`bat`, `bar`	2
`z`	None	0

If another `apple` were inserted, the `ap` count would become 3 and the `app` count would become 2. This is why insertion must preserve duplicates.

2.4 Correctness proof

For a node representing prefix `p`, an inserted word visits the node exactly when it begins with `p`. Incrementing once per visited node and per input entry makes `pass[v]` equal to the number of entries beginning with `p`.

A successful query reaches precisely the node for its prefix, so returning that node's count is correct. If a query edge is absent, no inserted entry has the required character path, so returning zero is correct.

2.5 Complexity and implementation notes

Building takes `O(W)` time and all queries together take `O(P)` time for the fixed lowercase alphabet. Trie storage is `O(W + 1)` nodes; a query costs `O(|prefix|)` time at most.

The lesson's `Trie.insert` and `Trie.count_prefix` provide these operations. Its implementation increments the root too, giving the empty prefix count `n`; local queries are nonempty. Its field is called `pass` in C++ and `passed` in Python; the lesson's introductory pseudocode calls it `cnt`. They express the same invariant.

In C++, zero is the root index and also the missing-child marker. Every allocated child has a positive index, so the two uses are unambiguous. Keep indices rather than references when growing the node vector. In Python, use an independent dictionary for each node. Reading the entire input at once additionally stores `O(W + P)` input characters; queries may instead be read one at a time.

2.6 Full C++ solution

```cpp
#include <bits/stdc++.h>
using namespace std;

class Trie {
    struct Node {
        array<int, 26> next{};
        int pass = 0;
    };
    vector<Node> nodes{Node{}};

public:
    void insert(const string& word) {
        int v = 0;
        nodes[v].pass++;
        for (char ch : word) {
            int c = ch - 'a';
            if (nodes[v].next[c] == 0) {
                int child = nodes.size();
                nodes[v].next[c] = child;
                nodes.push_back(Node{});
            }
```

```
            v = nodes[v].next[c];
            nodes[v].pass++;
        }
    }

    int count_prefix(const string& prefix) const {
        int v = 0;
        for (char ch : prefix) {
            int child = nodes[v].next[ch - 'a'];
            if (child == 0) {
                return 0;
            }
            v = child;
        }
        return nodes[v].pass;
    }
};

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);

    int n, q;
    if (!(cin >> n >> q)) {
        return 0;
    }
    Trie trie;
    for (int i = 0; i < n; i++) {
        string word;
        cin >> word;
        trie.insert(word);
    }
    while (q--) {
        string prefix;
        cin >> prefix;
        cout << trie.count_prefix(prefix) << '\n';
    }
}
```

2.7 Full Python solution

```
import sys


class Trie:
    def __init__(self):
        self.children = [{}]
        self.passed = [0]
```

```
    def insert(self, word):
        v = 0
        self.passed[v] += 1
        for ch in word:
            if ch not in self.children[v]:
                child = len(self.children)
                self.children[v][ch] = child
                self.children.append({})
                self.passed.append(0)
            v = self.children[v][ch]
            self.passed[v] += 1

    def count_prefix(self, prefix):
        v = 0
        for ch in prefix:
            if ch not in self.children[v]:
                return 0
            v = self.children[v][ch]
        return self.passed[v]


def main():
    tokens = iter(sys.stdin.read().split())
    n = int(next(tokens))
    q = int(next(tokens))
    trie = Trie()
    for _ in range(n):
        trie.insert(next(tokens))

    answers = []
    for _ in range(q):
        prefix = next(tokens)
        answers.append(str(trie.count_prefix(prefix)))
    print("\n".join(answers))


if __name__ == "__main__":
    main()
```

2.8 Common mistakes and useful tests

Test a query equal to a whole word, a query longer than every word, an absent edge, duplicate dictionary entries, and one dictionary word that prefixes another. A terminal-word counter would answer the wrong question: a word continuing below a node still contributes to that node's prefix count.

3 B. Palindrome Queries

3.1 Restatement and constraints

For each selected substring, determine whether it reads the same forward and backward. The text is unchanged between queries.

Both `n` and `q` may reach `200000`. Scanning or copying each selected substring takes `O(n*q)` time in the worst case, about forty billion characters. We need preprocessing and constant-time range comparisons. The lesson's rolling hash provides these operations.

3.2 Observation and algorithm

A palindrome is equal to its reverse. Build a rolling hash of `s` and another hash of the reversed full text. Every selected range corresponds to a mirrored range in that reversed text.

The API `get(L, R)` is 0-based and half-open. Input endpoints `l, r` are 1-based and inclusive, so set `L = l - 1` and `R = r`. The reversed range is `[n - R, n - L)`. The two ranges have the same length.

```
build the forward and reversed-text rolling hashes once
for each input query l, r:
    L = l - 1 and R = r
    compare forward.get(L, R) with backward.get(n - R, n - L)
    answer YES if both residues match, otherwise NO
```

`h[k]` stores the hash of the first `k` characters, so `h[0]` represents the empty prefix. This prefix-length indexing does not make the substring API 1-based. Both language implementations use the same half-open ranges.

3.3 Sample walkthrough

For `s = abacaba`, the reversed full text is also `abacaba`:

Input query	Forward range	Selected text	Mirrored range	Reversed text	Answer
`1 7`	`[0,7)`	`abacaba`	`[0,7)`	`abacaba`	YES
`2 4`	`[1,4)`	`bac`	`[3,6)`	`cab`	NO
`3 5`	`[2,5)`	`aca`	`[2,5)`	`aca`	YES
`1 3`	`[0,3)`	`aba`	`[4,7)`	`aba`	YES

The reverse of a selected substring is found in the mirrored range, not necessarily at the same endpoints in the reversed full text.

3.4 Correctness and collision model

The mirrored range contains exactly the selected substring's characters in reverse order. Thus, exact equality of these two strings is equivalent to the palindrome condition.

Identical strings always yield identical hash pairs, so every palindrome receives YES. If the pairs differ, the strings differ and NO is correct. A pair collision can produce a false YES for different strings. The two moduli reduce collision risk, but this is still a probabilistic comparison, not an exact equality proof. The reference follows the lesson's double-hash approach. A correct efficient exact method, such as Manacher preprocessing, is also a valid solution; performance tests reject quadratic scans, not alternative efficient algorithms.

3.5 Complexity and implementation notes

Preprocessing the two texts takes `O(n)` time and memory. Each query takes `O(1)` time, for `O(n + q)` total time. The Python reference buffers `O(q)` output strings in addition to the `O(n)` hash arrays.

Do not construct a substring inside the query loop and do not verify every hash match by rescanning the range: many long palindrome queries would then restore quadratic work. Create the forward and backward hash structures once, outside the loop. C++ needs 64-bit intermediate products; Python's modulo already normalizes negative residues.

3.6 Full C++ solution

```cpp
#include <bits/stdc++.h>
using namespace std;

class RollingHash {
    static constexpr long long MOD1 = 1000000007;
    static constexpr long long MOD2 = 1000000009;
    static constexpr long long BASE = 911382323;
    vector<long long> h1, h2, p1, p2;

public:
    explicit RollingHash(const string& s) {
        int n = s.size();
        h1.assign(n + 1, 0);
        h2.assign(n + 1, 0);
        p1.assign(n + 1, 1);
        p2.assign(n + 1, 1);
        for (int i = 0; i < n; i++) {
            int value = s[i] - 'a' + 1;
            h1[i + 1] = (h1[i] * BASE + value) % MOD1;
            h2[i + 1] = (h2[i] * BASE + value) % MOD2;
            p1[i + 1] = p1[i] * BASE % MOD1;
            p2[i + 1] = p2[i] * BASE % MOD2;
        }
```

```
    }

    // Hash of s[l:r], with 0-based, half-open endpoints.
    pair<long long, long long> get(int l, int r) const {
        long long a = (h1[r] - h1[l] * p1[r - l]) % MOD1;
        long long b = (h2[r] - h2[l] * p2[r - l]) % MOD2;
        if (a < 0) {
            a += MOD1;
        }
        if (b < 0) {
            b += MOD2;
        }
        return {a, b};
    }
};

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);

    string s;
    if (!(cin >> s)) {
        return 0;
    }
    string reversed = s;
    reverse(reversed.begin(), reversed.end());
    RollingHash forward(s);
    RollingHash backward(reversed);
    int n = s.size();

    int q;
    cin >> q;
    while (q--) {
        int l, r;
        cin >> l >> r;
        l--;  // Input [l, r] becomes the 0-based range [l - 1, r).
        bool same = forward.get(l, r) == backward.get(n - r, n - l);
        cout << (same ? "YES" : "NO") << '\n';
    }
}
```

3.7 Full Python solution

```
import sys


class RollingHash:
    MOD1 = 1_000_000_007
```

```
    MOD2 = 1_000_000_009
    BASE = 911_382_323

    def __init__(self, s):
        n = len(s)
        self.h1 = [0] * (n + 1)
        self.h2 = [0] * (n + 1)
        self.p1 = [1] * (n + 1)
        self.p2 = [1] * (n + 1)
        for i, ch in enumerate(s):
            value = ord(ch) - ord("a") + 1
            self.h1[i + 1] = (
                self.h1[i] * self.BASE + value
            ) % self.MOD1
            self.h2[i + 1] = (
                self.h2[i] * self.BASE + value
            ) % self.MOD2
            self.p1[i + 1] = self.p1[i] * self.BASE % self.MOD1
            self.p2[i + 1] = self.p2[i] * self.BASE % self.MOD2

    def get(self, l, r):
        # Hash of s[l:r], with 0-based, half-open endpoints.
        a = (
            self.h1[r] - self.h1[l] * self.p1[r - l]
        ) % self.MOD1
        b = (
            self.h2[r] - self.h2[l] * self.p2[r - l]
        ) % self.MOD2
        return a, b


def main():
    s = sys.stdin.readline().strip()
    forward = RollingHash(s)
    backward = RollingHash(s[::-1])
    n = len(s)
    q = int(sys.stdin.readline())
    answers = []
    for _ in range(q):
        l, r = map(int, sys.stdin.readline().split())
        l -= 1
        same = forward.get(l, r) == backward.get(n - r, n - l)
        answers.append("YES" if same else "NO")
    print("\n".join(answers))


if __name__ == "__main__":
    main()
```

3.8 Common mistakes and useful tests

Test single characters, both length parities, full-text ranges, boundary ranges, and a mismatch near the center. The small randomized oracle compares slices directly and does not use the hash solution to generate its answers.

Full-size tests include `200000` repeated characters with `200000` long palindrome queries, plus a string with one different character and a mix of symmetric, asymmetric, and boundary queries. The latter construction's answers follow from the exceptional character's position. These cases make per-query scanning and reversed substring copies exceed the time limit.

4 C. Distinct Substrings

4.1 Restatement and constraints

Count different nonempty string values appearing anywhere in `s`, once each, regardless of how often they occur or whether occurrences overlap.

The input length is at most `200000`. Enumerating all substring occurrences would produce up to `n * (n + 1) / 2` ranges. Storing their string values in a set is suitable only for tiny verification cases, not the local maximum input. Use the suffix automaton developed in Chapter 4 of the lesson.

4.2 Reuse the lesson's state invariant

A non-root state groups substrings with exactly the same set of ending positions, called `endpos`. Those strings are suffixes of one another and occupy a consecutive interval of lengths, with exactly one string at each length. The implementation stores the interval through:

```
longest represented length  = len[v]
shortest represented length = len[link[v]] + 1
number of strings in v      = len[v] - len[link[v]]
```

The suffix link identifies the next shorter group, where the ending-position set grows. It does not mean "the previous state created." State identifiers are not lengths. For example, in the lesson's `abcbc` graph, `link[6] = 7`, but `len[7] = 2`: state 6 contributes `5 - 2 = 3` strings.

The question counts string values, not occurrences. State `v` contributes its number of represented lengths, not the size of its `endpos` set.

4.3 Algorithm and the role of clones

Start with the empty automaton, append the input characters in order, then sum the contributions of every non-root state in the completed automaton. The lesson's `extend` and `count_distinct` implement these operations.

```
start with the root only
for each input character from left to right:
    extend the automaton by that character

answer = 0
for each final state except the root:
    answer += len[state] - len[link[state]]
report answer
```

A clone separates an old group when only its shorter strings acquire the new ending position. It inherits transitions and the old suffix link; the old state keeps the longer strings. Certain incoming character transitions are redirected to the clone, and both the old state and the new whole-text state receive a suffix link to it. Chapter 4's `ab` to `abb` example details those four updates.

Count clones just like other final non-root states. They own part of a group after a split. Excluding them would omit strings; treating their creation as an extra occurrence would also answer the wrong question. The sum above describes the final groups, not the order in which strings or states first appeared during construction.

4.4 Sample walkthrough: `ababa`

This sample creates no clones. Its completed groups are:

State	Strings represented	`endpos`	`len[v]`	`link[v]`	Contribution
1	`a`	`{1, 3, 5}`	1	0	`1 - 0 = 1`
2	`b, ab`	`{2, 4}`	2	0	`2 - 0 = 2`
3	`ba, aba`	`{3, 5}`	3	1	`3 - 1 = 2`
4	`bab, abab`	`{4}`	4	2	`4 - 2 = 2`
5	`baba, ababa`	`{5}`	5	3	`5 - 3 = 2`

For example, state 3 represents lengths 2 and 3. Its link reaches state 1, whose maximum length is 1, so its contribution is `3 - 1 = 2`.

The total is `1 + 2 + 2 + 2 + 2 = 9`. Reading the strings in these groups gives exactly the sample's nine values. Although `a` occurs three times, it contributes only one value. Different groups can contain strings of the same length; their string values, not their length intervals, are disjoint.

As a clone check, the lesson's `abb` has groups `a`, `ab`, `bb, abb`, and clone group `b`. They contribute `1 + 1 + 2 + 1 = 5`. The larger `abcbc` example has two clones and contributes 12, as shown in the lesson.

4.5 Correctness proof

By the lesson's SAM invariant, each distinct nonempty substring belongs to exactly one non-root state, and each state's strings occupy every length from `len[link[v]] + 1` through `len[v]`, once per length. The number of strings in that state is therefore `len[v] - len[link[v]]`.

The algorithm sums this size for all non-root states, including clones. Their groups partition the distinct nonempty strings, so every required string is counted once. The root contributes the empty string and is excluded. The resulting sum is the required answer.

4.6 Complexity and implementation notes

There are at most `2*n - 1` states for `n >= 2`; a one-character input has one root and one non-root state. Construction with the C++ maps takes `O(n log alphabet)` time, and Python dictionaries give expected `O(n)` time for the fixed lowercase alphabet. The final sum takes `O(n)` time, and storage is `O(n)` for this alphabet.

Call `count_distinct()` once after building the full text. The reusable method scans all states each time; calling it after every append would make that counting work quadratic.

Use `long long` for the C++ answer. The upper bound `n * (n + 1) / 2` is 20000100000 at `n = 200000`, exceeding a signed 32-bit integer. Python's integers already accommodate the result. The Python implementation calls maximum lengths `length`, while C++ calls the field `len`.

When implementing the structure, copy the clone's transition table rather than sharing mutable storage. In Python this means copying the dictionary. In C++, growing a vector can invalidate references: retain indices and copy the old state before appending the clone. Pattern-reading transitions and suffix links have separate roles; only the suffix links determine the lower boundary in this sum.

4.7 Full C++ solution

```cpp
#include <bits/stdc++.h>
using namespace std;

class SuffixAutomaton {
    struct State {
        map<char, int> next;  // Character transitions: extend to the right.
        int link = -1;        // Next group of shorter suffixes.
        int len = 0;          // Longest string in this group.
    };
    vector<State> states{State{}};
    int last = 0;

public:
    void extend(char ch) {
        // 1. Make a state for the new whole text and its new suffixes.
        int cur = states.size();
        states.push_back(State{});
        states[cur].len = states[last].len + 1;
        int p = last;

        // 2. Visit old suffix groups that have no ch continuation.
        while (p != -1 && !states[p].next.count(ch)) {
            states[p].next[ch] = cur;
            p = states[p].link;
        }

        // 3. Find the next shorter group for cur.
        if (p == -1) {
```

```
            // Case 1: ch is new; only the empty suffix was known.
            states[cur].link = 0;
        } else {
            int q = states[p].next.at(ch);
            if (states[p].len + 1 == states[q].len) {
                // Case 2: all of q is the next suffix group.
                states[cur].link = q;
            } else {
                // Case 3: split q; clone keeps its shorter strings.
                int clone = states.size();
                State copied = states[q];
                copied.len = states[p].len + 1;
                states.push_back(copied);

                // Redirect only the shorter incoming contexts.
                while (p != -1) {
                    auto it = states[p].next.find(ch);
                    // Every shorter suffix also has a ch transition.
                    assert(it != states[p].next.end());
                    if (it->second != q) {
                        break;
                    }
                    it->second = clone;
                    p = states[p].link;
                }
                // Both longer groups now have clone as suffix parent.
                states[q].link = clone;
                states[cur].link = clone;
            }
        }
        last = cur;
    }

    long long count_distinct() const {
        long long answer = 0;
        for (int v = 1; v < (int)states.size(); v++) {
            int parent = states[v].link;
            answer += states[v].len - states[parent].len;
        }
        return answer;
    }
};

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);

    string s;
    if (!(cin >> s)) {
        return 0;
    }
```

```
    SuffixAutomaton sam;
    for (char ch : s) {
        sam.extend(ch);
    }
    cout << sam.count_distinct() << '\n';
}
```

4.8 Full Python solution

```
import sys


class SuffixAutomaton:
    def __init__(self):
        self.next = [{}]
        self.link = [-1]
        self.length = [0]
        self.last = 0

    def extend(self, ch):
        # 1. Make a state for the new whole text and its new suffixes.
        cur = len(self.next)
        self.next.append({})
        self.length.append(self.length[self.last] + 1)
        self.link.append(0)
        p = self.last

        # 2. Visit old suffix groups that have no ch continuation.
        while p != -1 and ch not in self.next[p]:
            self.next[p][ch] = cur
            p = self.link[p]

        # 3. Find the next shorter group for cur.
        if p == -1:
            # Case 1: ch is new; only the empty suffix was known.
            self.link[cur] = 0
        else:
            q = self.next[p][ch]
            if self.length[p] + 1 == self.length[q]:
                # Case 2: all of q is the next suffix group.
                self.link[cur] = q
            else:
                # Case 3: split q; clone keeps its shorter strings.
                # Copy the dictionary so the two states are independent.
                clone = len(self.next)
                self.next.append(self.next[q].copy())
                self.length.append(self.length[p] + 1)
                self.link.append(self.link[q])
```

```
                # Redirect only the shorter incoming contexts.
                while p != -1:
                    # Every shorter suffix also has a ch transition.
                    assert ch in self.next[p]
                    if self.next[p][ch] != q:
                        break
                    self.next[p][ch] = clone
                    p = self.link[p]
                # Both longer groups now have clone as suffix parent.
                self.link[q] = clone
                self.link[cur] = clone

        self.last = cur

    def count_distinct(self):
        return sum(
            self.length[v] - self.length[self.link[v]]
            for v in range(1, len(self.next))
        )


def main():
    s = sys.stdin.readline().strip()
    sam = SuffixAutomaton()
    for ch in s:
        sam.extend(ch)
    print(sam.count_distinct())


if __name__ == "__main__":
    main()
```

4.9 Common mistakes and useful tests

Do not count states, occurrences, or only suffixes of the whole text. Do not subtract a suffix-link identifier instead of its maximum length, skip clones, or count the root.

Check `a` (1), `aaaa` (4), `abcd` (10), the sample `ababa` (9), `abb` (5), `abcbc` (12), and `abcbcd` (18). The last three connect directly to the lesson's split and extension examples. Long repeated and alternating texts help expose algorithms that accidentally enumerate substrings.

5 D. Unique Prefix Lengths

5.1 Restatement and constraints

For each input entry, find its shortest nonempty prefix shared by exactly one input entry. Equal words remain separate entries. A word that is a prefix of another entry has no answer, because every prefix it offers is shared with that other entry.

There are up to `200000` words and their total length `L` is at most `200000`. Test uniqueness against the entire list before producing answers; reporting an answer does not remove a word from the dictionary.

5.2 Observation and algorithm

Use the same trie invariant as A: `pass[v]` counts input entries beginning with the node's prefix. After inserting every word, a prefix is unique exactly when its node has `pass[v] = 1`.

Walk each word's path in order of increasing prefix length. The first node with count 1 supplies the shortest valid length. If no such node exists, answer -1. The root is never a candidate because its prefix is empty.

```
insert all input entries, maintaining pass counts

for each input entry in its original order:
    start at the root
    walk one character edge at a time
    stop at the first reached node with pass = 1
    report that prefix length, or -1 if no node qualifies
```

5.3 Sample walkthrough

For `cat`, `car`, `dog`, `do`, `cat`, the relevant paths have counts:

Word	Counts along its path	First unique prefix	Answer
`cat`	`c:3`, `ca:3`, `cat:2`	None	-1
`car`	`c:3`, `ca:3`, `car:1`	`car`	3
`dog`	`d:2`, `do:2`, `dog:1`	`dog`	3
`do`	`d:2`, `do:2`	None	-1
`cat`	Same counts as its other copy	None	-1

The node for `do` is not unique even though a word ends there: `dog` also passes through it. The two copies of `cat` must each receive an answer.

5.4 Correctness proof

Each dictionary entry visits exactly its nonempty prefix nodes during insertion. A node's count therefore equals the number of entries beginning with its prefix. A node qualifies exactly when this count is 1.

The second traversal visits candidate prefixes from shortest to longest. Consequently, its first qualifying node is the shortest valid prefix. If none qualifies, every candidate is shared and -1 is correct. Since the trie contains the complete list before any traversal, each answer uses all input entries and is unaffected by the order of reporting.

5.5 Complexity and implementation notes

Insertion takes `O(L)` time. Walking each word's path once takes `O(L)` additional time. Storage is `O(L)` for the trie and the input words, using fixed lowercase transitions in C++ or dictionaries in Python.

Reuse the lesson's trie storage and count invariant. The demonstrated public interface exposes `insert` and `count_prefix`; D additionally needs a walk that checks the count after each next character. Add that traversal inside the trie, or use the child/count arrays directly in your own implementation.

Do not call `count_prefix` separately for every prefix length. For a word of length `m`, restarting from the root would inspect `1 + 2 + ... + m = O(m^2)` characters. Creating a new substring for each prefix can add another quadratic cost. A single incremental walk avoids both.

The reusable trie also counts the root; D simply ignores that count. The problem-specific reference programs leave the root count unused and increment only reached non-root nodes. Both choices give the same answers for the nonempty prefixes considered here.

Keep counts of entries, not terminal markers and not distinct word values. Store words for the second pass and return answers in input order. In Python, create independent child dictionaries; in C++, retain node indices when growing the trie vector.

5.6 Full C++ solution

```cpp
#include <bits/stdc++.h>
using namespace std;

struct Node {
    array<int, 26> next{};
    int pass = 0;
};

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);

    int n;
    if (!(cin >> n)) {
```

```
        return 0;
    }
    vector<string> words(n);
    vector<Node> trie(1);

    for (string& word : words) {
        cin >> word;
        int node = 0;
        for (char ch : word) {
            int c = ch - 'a';
            if (trie[node].next[c] == 0) {
                int child = trie.size();
                trie[node].next[c] = child;
                trie.push_back(Node{});
            }
            node = trie[node].next[c];
            trie[node].pass++;
        }
    }

    for (const string& word : words) {
        int node = 0;
        int answer = -1;
        for (int i = 0; i < (int)word.size(); i++) {
            int c = word[i] - 'a';
            node = trie[node].next[c];
            if (trie[node].pass == 1) {
                answer = i + 1;
                break;
            }
        }
        cout << answer << '\n';
    }
}
```

5.7 Full Python solution

```
import sys


def main():
    tokens = sys.stdin.buffer.read().split()
    if not tokens:
        return
    n = int(tokens[0])
    words = [word.decode() for word in tokens[1:n + 1]]
    children = [{}]
    passed = [0]
```

```
    for word in words:
        node = 0
        for ch in word:
            if ch not in children[node]:
                child = len(children)
                children[node][ch] = child
                children.append({})
                passed.append(0)
            node = children[node][ch]
            passed[node] += 1

    answers = []
    for word in words:
        node = 0
        answer = -1
        for length, ch in enumerate(word, 1):
            node = children[node][ch]
            if passed[node] == 1:
                answer = length
                break
        answers.append(str(answer))
    print("\n".join(answers))


if __name__ == "__main__":
    main()
```

5.8 Common mistakes and useful tests

Test identical words, a word that prefixes another, long shared prefixes, and one nonempty word by itself (answer 1). For `do` and `dog`, the answers are -1 and 3. For `app` and `apple`, they are -1 and 4. For `app` and `ape`, both answers are 3. Repeated identical one-letter entries all need -1.

Answering before all words have been inserted, deduplicating entries, checking only terminal markers, or accepting the root would break the required uniqueness condition.

6 Testing discussion

Use independent small oracles before trying large cases:

- A: count entries satisfying `word.startswith(prefix)` for each query.
- B: compare the selected slice with its reversal.
- C: enumerate nonempty substrings into a set for tiny texts, then take its size.
- D: scan prefix lengths in increasing order and count matching input entries; keep duplicates in that count.

For C, test after appending characters as well as on complete texts. This checks the lesson's `extend` operation, particularly clones, independently of the final sum. The exercise itself needs only the completed text's answer.

Large inputs serve a different purpose: long shared trie paths exercise storage and traversal; many long palindrome ranges reject per-query scans; long strings with many distinct substrings exercise time, memory, and 64-bit counts. Small random tests establish behavior, while maximum-size cases check that the chosen complexity fits.

Run the reference solutions in both languages with:

```
CP_TARGET=solution python3 sections/44_string_algorithms_ii/check.py --keep-going
```

7 What to carry forward

State exactly what a node groups before choosing its counter. Trie nodes group dictionary entries by prefix; SAM states group substring values by ending positions. A word ending at a trie node does not make its prefix unique, and several occurrences of a substring do not create several values to count. Keep input endpoints separate from the rolling hash's half-open API. For the local palindrome problem, preprocess range hashes instead of scanning each query; keep the exact scan as a small-input oracle.